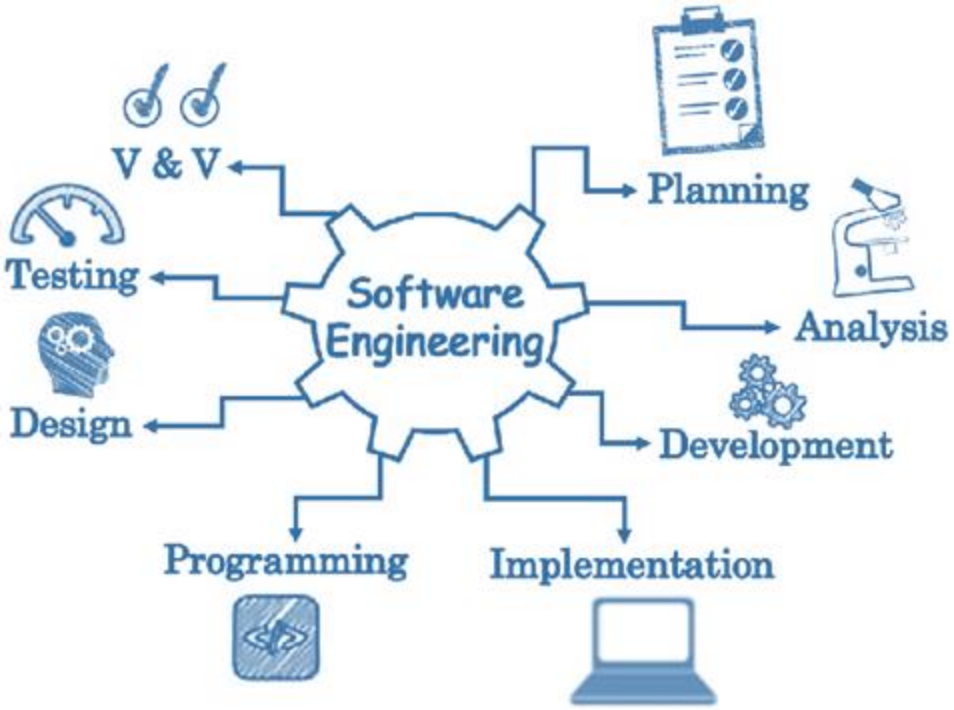
Software Engineering (CMP 519.3)



- Instructor
- Birodh Rijal

birodh.rijal@eemc.edu.np

Unit VII: Configuration Management and DevOps Tools (4 Hours)

7.1 Configuration Management Planning

- Importance of Configuration Management
- Configuration Items and Baselines

7.2 Change Management

- Managing Change Requests
- Version and Release Management

7.3 System Building

- Build Automation
- Continuous Integration Practices

7.4 CASE Tools for Configuration Management

- Overview of Tools like Git and Subversion

7.5 DevOps Tools in Configuration Management

Advanced Use of Jenkins, Docker, and Kubernetes

Configuration Management

- **Configuration management** is the discipline of managing and controlling change in the evolution of software systems [IEEE Std. 1042-1987].
- Configuration management systems automate the identification of versions, their storage and retrieval, and supports status accounting.
- Configuration management includes the following activities:
  - ***Identification of configuration items***. Components of the system and its work products and their versions are identified and labeled uniquely. Developers identify configuration items after the *Project Agreement* (Section 14.3.6), once the principal deliverables and components of the system are agreed on. Developers create versions and additional configuration items as the system evolves.
  - ***Change control***. Changes to the system and releases to the users are controlled to ensure consistency with project goals. Change control can be done by the developers, by management, or by a control board, depending on the level of quality required and the rate of change.
  - ***Status accounting***. The status of individual components, work products, and change requests are recorded. This allows developers to distinguish versions more easily and track issues related to changes. This also allows management to track project status.
  - ***Auditing.*** Versions selected for releases are validated to ensure the completeness, consistency, and quality of the product. Auditing is accomplished by the quality control team.

Configuration Management

Configuration Management Concepts

- A ***configuration item*** is a work product or a piece of software that is treated as a single entity for the purpose of configuration management.
- A composite of configuration items is defined as a ***configuration management aggregate*** (CM aggregate).
- A ***change request*** is a formal report issued by a user or a developer to request a modification in a configuration item.
- A ***version*** identifies the state of a configuration item or a configuration aggregate at a well-defined point in time.
- Versions that are intended to coexist are called ***variants***.
- A ***promotion*** is a version that has been made available to other developers in the project.
- A ***release*** is a version that has been made available to the client or the users.
- A ***software library*** stores versions and provides facilities to track the status of changes.
- A ***repository*** is a library of releases.
- A **master directory** is a library of promotions.

Configuration Management

Configuration Management Concepts

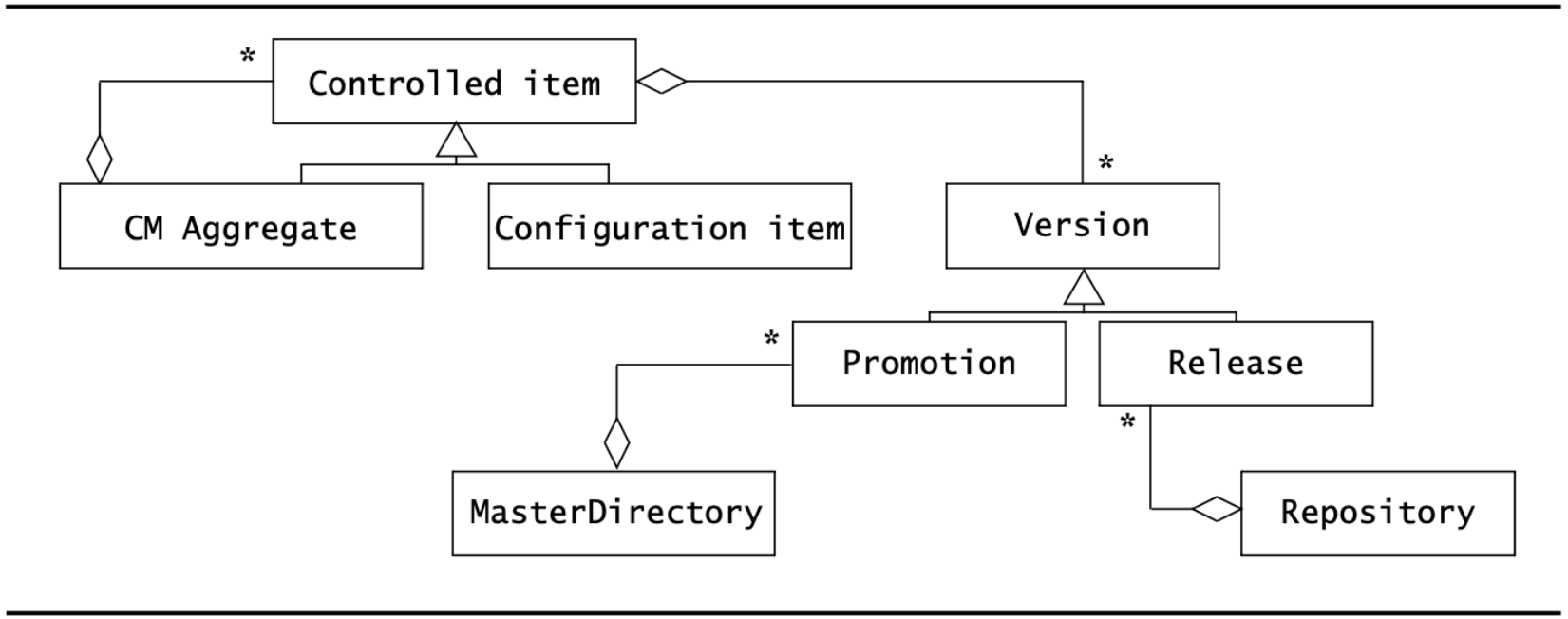


Figure 13-1 Configuration management concepts (UML class diagram).

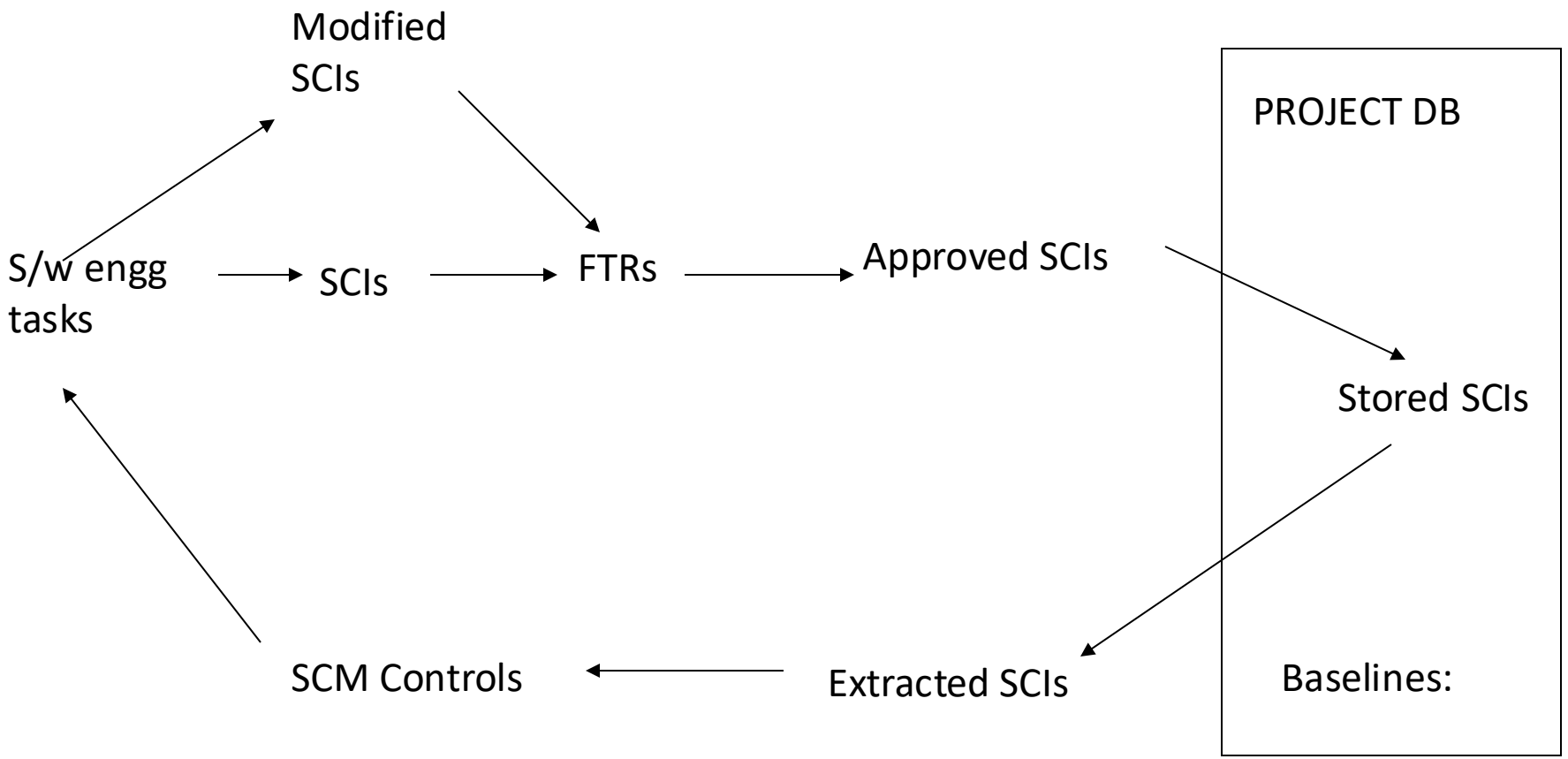
Configuration Management

Configuration Management Concepts : Baseline

- IEEE defines baseline as:
  - A specification or product that has been formally reviewed and agreed upon, that thereafter serves as basis for further development, and that can be changed only through formal change control procedures.
- Before a s/w configuration item becomes a baseline, changes may be made quickly & informally.
- However, once it becomes a baseline, formal procedures must be applied to evaluate & verify every change.
- Refer to figure 27.1 to see steps for forming a baseline.

Configuration Management

Configuration Management Concepts : Baseline



Configuration Management

Configuration Management Concepts : Configuration Object

- SCIs are organized to form configuration objects.
- A config. object has a name, attributes and is related to other objects.
- Example:
  - Test Specification – Config. Object
  - It comprises of SCIs like Test Plan, Test Procedure, Test Cases
  - It is related to config. object SourceCode
- Config. Objects are related to other cong. objects, the relationship is either compositional (part-of relationship) or interrelated.
- In related config. objects, if one of them changes, the other one has to be changed.

Configuration Management

Configuration Management Concepts : SCM Repository

- In early days of s/w engg, software configuration items were maintained as paper documents (or punch cards), placed in files and folders.
- The problems of this approach were:
  - Finding a configuration item when it was needed was often difficult
  - Determining which items were changed, when and by whom was often challenging
  - Constructing a new version of existing program was error-prone & time consuming
  - Describing complex relationships among configuration items was virtually impossible
- Today SCIs are maintained in a project database or repository.
- It acts as the center for accumulation and storage of SCIs

Configuration Management

Configuration Management Concepts : Role of SCM Repository

- SCM Repository is a set of mechanisms & data structures that allow a software team to manage change in an effective manner.
- It provides functions of a DBMS but in addition has the following functions:
  - Data integrity
  - Info sharing
  - Tool integration
  - Data integration
  - Methodology enforcement
  - Document standardization

Change Management

- Change requests can range from correcting a typo in a menu label to reimplementing a major subsystem to address performance issues.
- Change requests also vary in their timing: a request for additional functionality during the requirements review can lead to modifying the draft RAD, whereas the same request made during testing can additionally lead to major subsystem surgery.

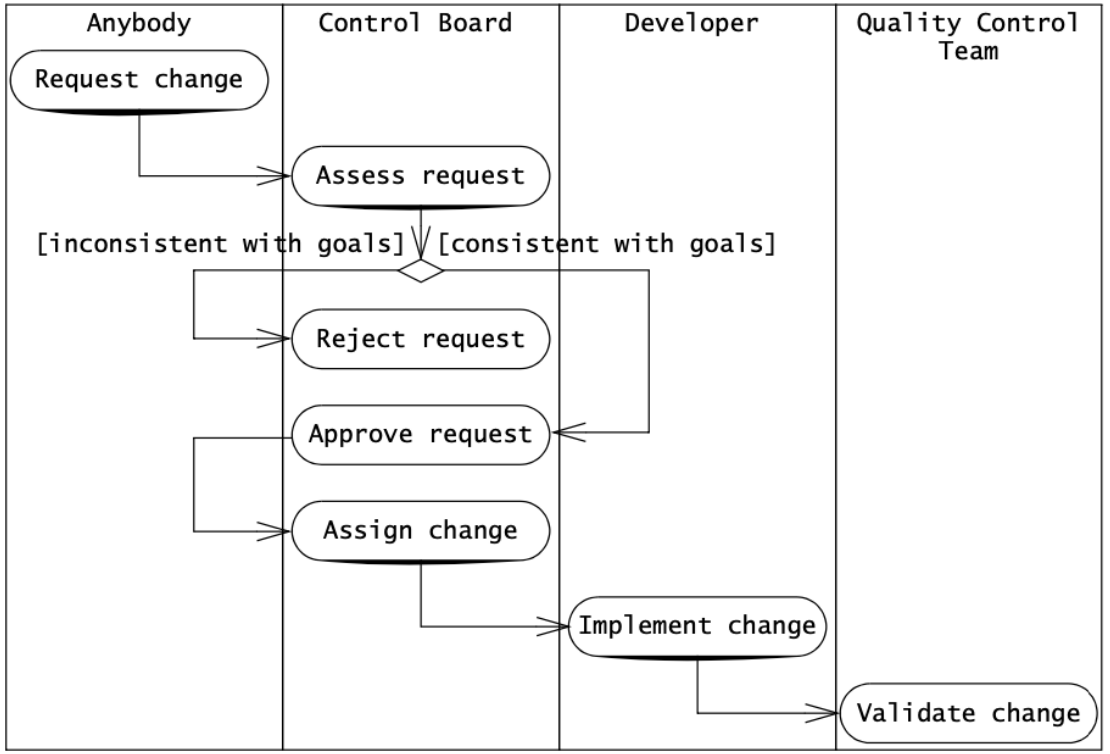


Figure 13-14 An example of change management process (UML activity diagram).

Version Control

- *Combines procedures and tools to manage the different versions of configuration objects created during the software process.*
- Version
  - A particular version is an instance of a system that differs in some way from other instances (difference may be enhanced performance, functionality or repaired faults).
  - A software component with a particular version is a collection of objects.
  - A new version is defined when major changes have been made to one or more objects

Version Control

- Object 1.0 undergoes revision
  - Object 1.0 -> Object 1.1
- If minor changes/corrections introduced to Object 1.1
  - Object 1.1.1 & 1.1.2 (variants)
- Functional changes, new modules added to Object 1.1 can lead to
  - Object 1.1 ->Object 1.2
- Major Technical Change
  - Object 1.0 ->Object 2.0

Change Control

- A Change request is submitted and evaluated to assess technical merit and impact on the other configuration objects and budget
- Change report contains the results of the evaluation
- Change control authority (CCA) makes the final decision on the status and priority of the change based on the change report
- Engineering change order (ECO) is generated for each change approved (describes change, lists the constraints, and criteria for review and audit)

Change Control (Contd.)

- Object to be changed is checked-out of the project database subject to access control parameters for the object
- Modified object is subjected to appropriate SQA and testing procedures
- Modified object is checked-in to the project database and version control mechanisms are used to create the next version of the software
- Synchronization control is used to ensure that parallel changes made by different people don't overwrite one another

System Building

- Refers to the process of **integrating all software components**, including source code, libraries, and configuration files, into a working system.
- Typically involves:
  - **Compiling** the source code.
  - **Linking** various modules and libraries.
  - Generating the **final build** or executable.
- Key tools: Build automation tools like **Maven**, **Gradle**, or **Make**.

Build Automation

- Automated process of compiling, linking, testing, and packaging software.
- Eliminates manual errors and increases efficiency.
- Key areas:
  - Source Code Compilation
  - Dependency Management
  - Automated Testing
  - Build Scripting
  - Version Control Integration

Why is Build Automation Important?

- **Efficiency** – Speeds up the development cycle.
- **Consistency** – Ensures reproducible and reliable builds.
- **Scalability** – Supports large projects with multiple dependencies.
- **CI/CD Integration** – Works with Continuous Integration/Continuous Deployment.
- **Early Error Detection** – Automates testing and feedback.

Components of Build Automation

1. **Source Code Compilation** – Converts code into executables.
2. **Dependency Management** – Ensures correct libraries are included.
3. **Automated Testing** – Runs unit and integration tests.
4. **Build Scripting** – Automates build steps with scripts.
5. **Version Control Integration** – Automates builds upon code changes.

Build Automation Tools

Tool	Description
Maven	Java-based build and dependency management.
Gradle	Flexible build system (e.g., Android).
Jenkins	Automates CI/CD pipelines.
GitHub Actions	Automates builds from GitHub repositories.
Bazel	Google’s scalable build system.
CMake	Cross-platform build system (C++).
Make	Traditional Unix build tool.

Continuous Integration (CI) & Build Automation

- CI ensures **frequent code integration and automated testing**.
- **Build automation is a key part of CI**.
- **Popular CI tools:**
  - Jenkins
  - GitHub Actions
  - GitLab CI
  - Travis CI
  - CircleCI

Best Practices for Build Automation

- **Use version control** (Git, SVN) for build scripts.
- **Ensure builds are repeatable** (store dependencies in configuration files).
- **Run automated tests** before deployment.
- **Use containerization** (Docker) to ensure environment consistency.
- **Monitor build performance** and optimize bottlenecks.

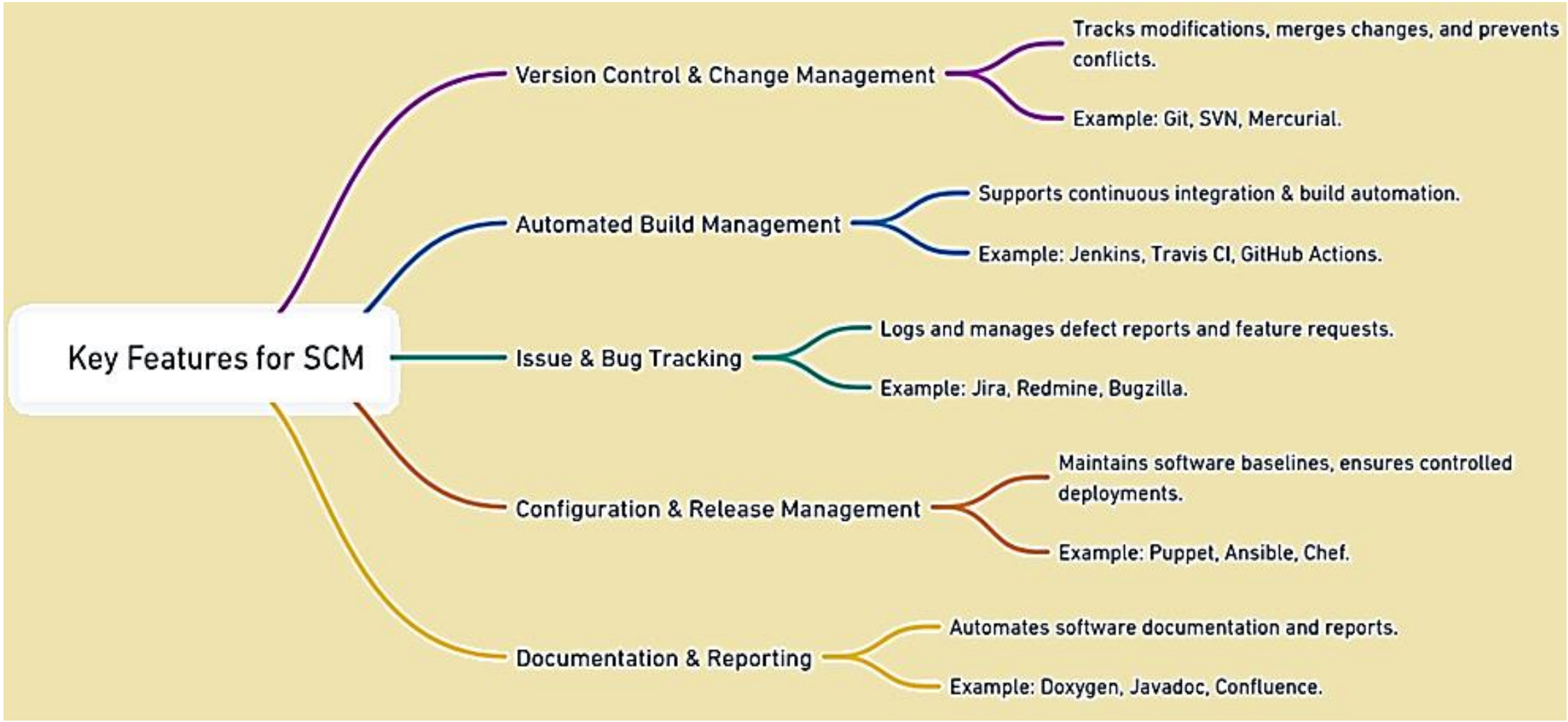
CASE Tools

- CASE (Computer-Aided Software Engineering) tools assist in **software development, maintenance, and configuration management**.
- **Purpose**: Automates **documentation, coding, testing, and version control**.
- **Types**:
  - **Upper CASE** – Supports planning & design.
  - **Lower CASE** – Supports coding, testing, and maintenance.
  - **Integrated CASE (I-CASE)** – Supports the full development lifecycle.

CASE Tools in Configuration Management

- **Automates tracking of software changes.**
- **Facilitates version control** and release management.
- **Ensures consistency across development teams.**
- **Supports automated testing and debugging.**
- **Manages software artifacts like requirements, source code, and documentation.**

Key features of CASE tools for SCM



Popular CASE Tools for Configuration Management

Tool	Category	Description
Git	Version Control	Tracks changes, branches, merges code.
SVN (Subversion)	Version Control	Centralized version control system.
Jenkins	Build Automation	Automates builds and CI/CD pipelines.
Travis CI	Build & Testing	Supports automated software testing.
Jira	Issue Tracking	Manages change requests and bugs.
Ansible	Configuration Mgmt	Automates deployment & infrastructure management.
Docker	Environment Mgmt	Ensures consistency across environments.

Challenges of CASE Tools in SCM

- **Learning Curve** – Some tools require training.
- **Integration Issues** – Compatibility with existing infrastructure.
- **Cost Overheads** – Premium tools may have licensing fees.
- **Security Concerns** – Misconfigurations can expose vulnerabilities.
- **Scalability** – Some tools struggle with large-scale projects.

DevOps & Configuration Management

- **What is DevOps?**
  - A **cultural and technical approach** that integrates development and operations.
  - Focuses on **continuous integration, deployment, and monitoring**.
- **Role of Configuration Management in DevOps**
  - Automates **software provisioning, infrastructure setup, and system maintenance**.
  - Ensures **consistent environments across development, testing, and production**.

Why Use DevOps Tools in Configuration Management?

- **Automation** – Reduces manual configurations and errors.
- **Scalability** – Manages large-scale applications efficiently.
- **Consistency** – Ensures software behaves the same across environments.
- **Rapid Deployment** – Accelerates release cycles.
- **Monitoring & Security** – Ensures compliance and system health.

DevOps Tools for Configuration Management

Tool	Category	Purpose
Jenkins	CI/CD Automation	Automates builds, tests, and deployments.
Docker	Containerization	Packages applications with dependencies.
Kubernetes	Orchestration	Manages containerized applications at scale.
Ansible	Configuration Mgmt	Automates infrastructure provisioning.
Terraform	Infrastructure as Code	Automates cloud infrastructure management.
GitHub Actions	CI/CD Automation	Automates workflows for version control.

Comparing Jenkins, Docker, and Kubernetes

Feature	Jenkins	Docker	Kubernetes
Primary Function	CI/CD Automation	Containerization	Container Orchestration
Deployment Type	Pipelines & Jobs	Containers	Multi-Container Apps
Scalability	Parallel Jobs	Lightweight Containers	Auto-Scaling
Configuration	Jenkinsfile	Dockerfile	YAML Manifests
Integration	Git, Kubernetes, Terraform	Jenkins, Kubernetes	Docker, CI/CD Pipelines